

DIA 1 . Tiempo dedicado: 3h

OBJETIVO

Crear la base del proyecto, organizar la estructura inicial , el control de versiones y la sincronización con GitHub

ACCIONES

1. Creación de la carpeta principal del proyecto

Se crea la carpeta raíz del proyecto con el nombre **proyecto-hospital** para disponer de un directorio principal sobre el que organizar toda la estructura del proyecto.

2. Creación de la estructura inicial de carpetas

Dentro de la carpeta principal se crea la estructura base del proyecto, siguiendo una estructura MVC desde el inicio, separando claramente controladores, modelos, vistas, núcleo del sistema, base de datos, etc.

- **app**

Contendrá el código principal de la aplicación. En ella se almacenarán los componentes que implementarán la lógica del sistema siguiendo una arquitectura **MVC**.

Dentro de ella se incluirán, a su vez, las subcarpetas **controllers**, **models**, **views**, **core** y **config**, que permiten organizar la aplicación siguiendo el patrón Modelo-Vista-Controlador (MVC), separando la lógica del sistema de los recursos.

- **public**

Contendrá los archivos accesibles directamente desde el navegador, separándolos del resto del código interno del sistema.

Aquí se encuentra el archivo index.php, que actúa como punto de entrada de la aplicación. También otros recursos públicos como archivos CSS, javaScript o imágenes.

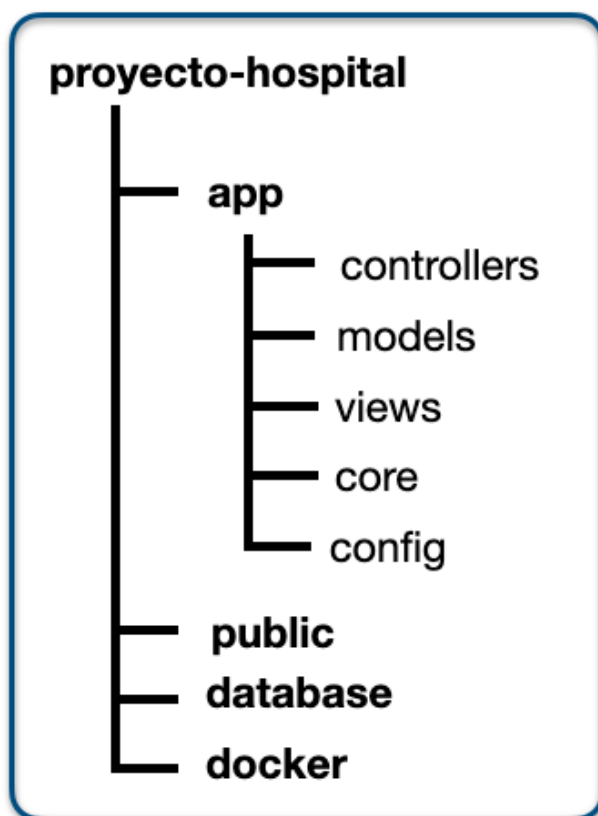
- **database**

Almacenará los archivos relacionados con la base de datos. Centraliza y versiona la estructura de la base de datos dentro del proyecto. En ella se guardan los scripts SQL necesarios para crear la estructura de la base de datos.

- **docker**

Almacenará los archivos relacionados con los contenedores de Docker.

Estructura inicial



3. Creación de los archivos base del proyecto

- **public/index.php**

Punto de entrada de la aplicación.

- **README.md**

Archivo de documentación general del proyecto.

- **.gitignore**

Archivo para indicar a Git qué elementos no deben subirse al repositorio

4. Creación de un contenido inicial de prueba

En el archivo public/index.php se añade un texto para verificar más adelante que la aplicación carga correctamente en el navegador.

```
<?php  
Echo "Celcare está funcionando";
```

5. Inicialización del repositorio Git

Desde la terminal de Visual Studio Code, en la raíz del proyecto se ejecuta **git init** para inicializar el control de versiones del proyecto y permitir el seguimiento de cambios desde el primer día.

Se configura el archivo **.gitignore** con una lista básica de elementos que no deben subirse al repositorio, archivos innecesarios, temporales o específicos del sistema operativo.

```
Vendor/  
node_modules/  
.env  
.DS_Store
```

Se realiza el primer **commit** para guardar el primer estado estable del proyecto en el historial de versiones

```
git add .  
git commit -m "Estructura inicial del  
proyecto Celcare"
```

6.Creación y vinculación del repositorio remoto en GitHub

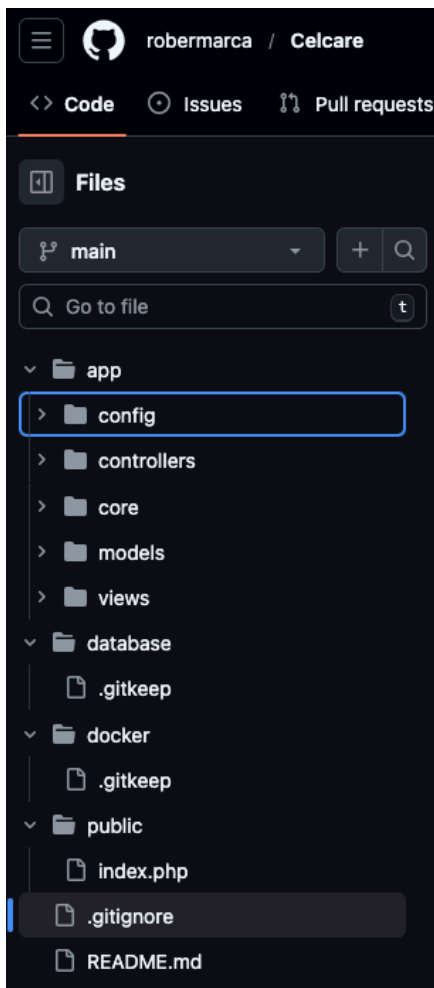
Se crea un repositorio remoto en GitHub llamado **Celcare** para poder disponer de una copia remota del proyecto y permitir el seguimiento del desarrollo mediante control de versiones distribuido.

Se conecta el repositorio local con el remoto mediante el comando:

```
git remote add origin https://github.com/robermarca/Celcare.git
```

Se configura la rama principal y se sube el proyecto.

```
git branch -M main  
git push -u origin main
```



Para poder visualizar todas las carpetas en GitHub, se añaden archivos **.gitkeep** dentro de las carpetas vacías para forzar su inclusión en el repositorio.

Dia 2 . Tiempo dedicado: 2h

OBJETIVO

Dejar funcionando el entorno de desarrollo con **Docker + PHP + PostgreSQL**

ACCIONES

1. Comprobación de Docker

```
docker --version (Docker version...)  
docker compose version (Docker compose version ...)
```

Comprobar que Docker está instalado y funcionando correctamente en el equipo.

En la terminal, dentro del proyecto, ejecutar:

Esto verifica que el entorno de contenedores está disponible antes de crear la configuración del proyecto.

2º.- Crear archivo docker-compose.yml

En la raíz del proyecto, al mismo nivel que las carpetas app, database, public, etc. crear:

docker-compose.yml

Esta acción define los servicios necesarios para ejecutar la aplicación en los contenedores.

Se configuran dos servicios:

- **PostgreSQL** , como sistema de gestor de bases de datos
- **PHP + Apache**, como entorno para ejecutar la aplicación web

Contenido del archivo

```
services:
  postgres:
    image: postgres:15
    container_name: celcare_postgres
    restart: always
    environment:
      POSTGRES_DB: celcare
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  php:
    image: php:8.2-apache
    container_name: celcare_php
    restart: always
    ports:
      - "8080:80"
    volumes:
      - ./public:/var/www/html

volumes:
  postgres_data:
```

Se utiliza la imagen oficial de **PostgreSQL 15** y la imagen oficial de **PHP 8.2** con Apache.

La aplicación se expone en el navegador a través del puerto:

`http://localhost:8080`

Para evitar la pérdida de datos al reiniciar los contenedores, se configura un volumen Docker llamado **postgres_data**, que permite persistir la información de la base de datos PostgreSQL fuera del contenedor.

3°.- Arrancar el entorno

Desde la carpeta del proyecto ejecutar

`docker compose up`

Docker descarga automáticamente las imágenes necesarias de PostgreSQL y de PHP + Apache.

Esta primera ejecución de ***docker compose up*** tarda más tiempo porque Docker debe descargar las imágenes necesarias.

Después crea :

- la red interna del proyecto
- el volumen de persistencia
- los contenedores celcare_postgres y celcare_php

El entorno queda levantado correctamente y accesible desde el navegador.

4°.- Verificar el funcionamiento

Arrancados los contenedores, se comprueba el funcionamiento accediendo a:

`http://localhost:8080`

La aplicación mostró correctamente el mensaje de prueba definido en el index.php. Esto confirma que tanto Apache como PHP están funcionando y que Docker está sirviendo correctamente el proyecto.

```
<?php  
echo "Celcare está funcionando";
```

DIA 3 . Tiempo dedicado:

OBJETIVOS

ACCIONES